Ntoetse Selomo

Assembler lead · GitHub: ntoetseselomo-crypto · Student no. 202321292

Responsible for the assembler that turns LIMO programs into 32-bit machine words.

Files and document sections

File / section	Content
`src/assembler.js`	Parsing helpers, error handling, two-pass assemble(), encoders, hex and binary output, format layouts.
`docs/M2-design.md §8`	Description of the two passes, formulas and error rules.
`examples/*.limo`	The five example programs the assembler accepts.

Work allocated

- In the first pass, the program removes comments marked with # or ;. It reads optional labels, records the byte address of each label, and moves the program counter forward by 4 for each instruction.
- During the second pass, it figures out the format of each instruction and organizes the fields into 32 bits. For R-format, it uses rd, rs1, rs2, funct. For I, S, and B formats, it includes the opcode, two registers, and a 14-bit immediate value.
- The branch displacement is calculated as (label - (PC + 4)) / 4, measured in words.
- Operand parsing: register names (r0–r15, n1–n13, lefela, sp, mokokotlo, ra), decimal and hexadecimal numbers, offset(register) memory operands, range check of −8192 to 8191.
- . It provides plain-language error messages with line numbers, and assembling halts at the first error. Curly apostrophes get flagged with a helpful message.
- Output for each instruction: pc, source line, format, fields, word, hex and binary, plus the label table, used by the page and by the M3 pipeline.